Project Documentation

AI Software Engineering Assistant

A multi agent system for automated bug investigation, fixing, and pull request generation

Introduction

This document walks through everything about the AI Software Engineering Assistant project, what it is, why it exists, how it's built, what it took to get it working, and what it can and can't do right now. The short version is that it's a multi agent AI system that takes a real GitHub issue and carries it all the way from a raw bug report to an actual, human approved pull request, using six specialized agents that each handle one part of that process. The longer version is basically the rest of this document.

Motivation

I wanted to build something that went past the usual capstone pattern of one agent answering one question. A lot of AI coding tools are really just a single model responding to a single prompt inside an editor, and while that's useful, it doesn't reflect how software actually gets fixed in a real team. A real bug fix involves understanding the report, figuring out where in the codebase it probably lives, drafting a fix, having someone review that fix, testing it, writing down what changed, and then getting it approved before it goes anywhere near production.

So the motivation here was to actually model that whole process, not shortcut it. Each step became its own agent with its own job, connected together by a system that could pass information between them reliably, loop back when something wasn't good enough, and, critically, stop and wait for a real person before doing anything that would actually change a repository.

What the system does

Here's the full path a single issue takes through the system, from start to finish.

You give the system a repository owner, a repository name, and an issue number. It fetches that real issue directly from GitHub, not a mock or a sample, the actual issue text as written by whoever reported it.

A Requirements Analysis agent reads that raw text and turns it into something structured, classifying it as a bug, a feature, or a chore, writing a one sentence summary, listing out acceptance criteria, and estimating how complex the fix is likely to be. This step matters more than it looks like it should, because every agent after this one builds on what this agent decided.

If the issue was classified as a bug, a Bug Investigation agent takes over. Before it says anything, it clones the actual target repository and indexes its code into a vector database, then retrieves the pieces of code most relevant to the issue and uses those, not general knowledge, to reason about what's probably causing the problem and which files are likely involved.

A Coding Assistant agent then proposes an actual fix, still grounded in that same retrieved code, and attaches a risk level along with a flag saying whether it thinks a human should look closely at this one before it goes further.

A Code Reviewer agent evaluates that fix. If it's not good enough, the system automatically sends it back to the Coding Assistant with the reviewer's feedback attached, and the Coding Assistant tries again. This can happen up to a limited number of times, after which the system stops trying to resolve it purely on its own and just flags it for a human, rather than looping endlessly hoping for a better result.

Once review is actually settled, whether that's a genuine approval or hitting the revision limit, two agents kick off at the same time. A Testing Agent writes an actual, self contained pytest file and runs it for real in a sandboxed process, not just generating text that resembles a test, and if that test fails because of a mistake in the generated test code itself, it takes that failure and tries writing the test again. Alongside it, a Documentation Writer agent produces a changelog entry, inline code comments explaining the change, and a short readme style summary.

At this point, the pipeline stops completely and shows everything it produced. Nothing happens automatically past here. A person has to look at what the system did and explicitly click approve. Only once that happens does the system move into real action, forking the target repository into your account if you don't already own it, creating a new branch, committing the actual proposed fix as a real file, and opening a genuine pull request with a full description linking back to the original issue.

Every fix that gets approved also gets embedded and stored in a separate memory collection, so the next time a similar issue comes through, the Bug Investigation agent can pull up what was done before and use it as additional context.

System architecture

The whole pipeline is built as a graph rather than a straight script, using LangGraph. Every agent is a node in that graph, and every node reads from and writes to one shared, typed state object that carries the entire history of the run, the issue text, the requirements, the investigation, the proposed fix, the review outcome, the test results, all of it, forward through the pipeline. Instead of each agent needing to be re given context by hand, everything downstream automatically has access to everything that happened before it.

Routing between agents is handled by conditional edges rather than nested if statements buried in application code. The graph itself decides, based on the current state, whether to route a fresh issue through Bug Investigation or skip straight to Coding, whether a rejected fix goes back to Coding Assistant or gets escalated, and when to fan out into Testing and Documentation running in parallel. This makes the actual control flow of the system something you can look at directly in the graph definition, rather than something buried across a dozen conditional branches.

Around that core agent graph sit four supporting layers. A retrieval layer clones and indexes whatever target repository is being worked on, into its own separate vector collection, so the system isn't limited to one hard coded demo codebase and can genuinely be pointed at any public repository. A memory layer persists resolved issues so the system's knowledge compounds across runs instead of resetting every time. A shared utilities layer handles the messy reality of working with language models, parsing and repairing JSON output, retrying calls that come back empty, and applying sensible defaults when a model skips a field it was supposed to return. And an actions layer turns an approved fix into real, verifiable GitHub activity.

The agents, in depth

Requirements Analysis is the entry point after the issue is fetched. Its only job is turning unstructured text into structured data the rest of the system can rely on. It doesn't touch code, it doesn't propose anything, it just reads and classifies.

Bug Investigation only runs for issues classified as bugs. Its defining feature is that it doesn't reason in a vacuum. Before it produces any output, it queries a vector index built from the actual target repository's real source files, and only then does it reason about likely causes, explicitly asked to reference real file paths and code it was actually shown rather than inventing plausible sounding guesses.

Coding Assistant takes the investigation's findings and drafts an actual fix. Like the investigation agent, it's given real retrieved code as context rather than working from the issue description alone, and it's explicitly asked to reference real function and file names

where it can. It also makes a judgment call on risk level and whether human review feels necessary, which feeds directly into the human approval gate later.

Code Reviewer is the first place self correction shows up. It doesn't just rubber stamp whatever Coding Assistant produces, it can genuinely reject a fix, explain why, and send it back. This loop is bounded, capped at a small number of revisions, specifically so the system can't get stuck cycling forever between rejection and resubmission. Once that cap is hit, the system treats the issue as resolved for pipeline purposes but flags it as needing human review, which is a deliberate design choice, an unresolved disagreement between agents should become a human's problem, not an infinite loop.

Testing Agent is the most operationally complex of the six, because it doesn't just generate text, it actually executes code. It writes a complete, self contained pytest file, meaning it can't rely on importing anything from the real target project, since that project's actual dependencies aren't installed in this environment, and it runs that file in a real subprocess sandbox, capturing genuine pass or fail results. If a test fails specifically because of a mistake in the generated test code itself, rather than a real disagreement about behavior, the agent feeds that failure output back into a second generation attempt, effectively debugging its own test.

Documentation Writer is the simplest of the six in terms of what it does, but it runs at the same time as Testing Agent rather than after it, since there's no reason to make one wait on the other.

Technology stack and why each piece was chosen

Python was the natural choice given the surrounding ecosystem of tools this project depends on.

LangGraph was chosen specifically because it treats multi agent systems as explicit state graphs rather than implicit chains of function calls, which made the branching and looping behavior this project needed, conditional routing, rejection loops, parallel fan out, dramatically easier to build and reason about than hand rolling the same logic with plain Python control flow.

Groq was chosen as the LLM provider mainly for speed and its free tier. Partway through the project, the model originally used, Llama 3.3 70B, was deprecated by Groq, which forced a real migration mid build to Groq's currently recommended replacement, an open weight OpenAI model called gpt oss 20b. That migration is discussed more in the challenges section below, since it surfaced a genuinely subtle bug that took real investigation to track down.

ChromaDB was chosen as the vector store because it's simple to run locally with zero external hosting cost, and it comfortably supports the two separate uses this project needed

from it, retrieval over target codebases and long term memory of resolved issues, as two independent collections.

Sentence Transformers, specifically a small, fast embedding model, handles generating the actual vector embeddings locally, which avoids needing a separate paid embedding API and keeps the whole project genuinely free to run.

The GitHub REST API is used directly through plain HTTP calls rather than a heavier SDK, both to fetch the original issue and, after approval, to perform the real fork, branch, commit, and pull request creation.

pytest was the obvious choice for test execution, since the goal was genuine pass or fail results rather than an LLM's opinion of whether a test would probably pass.

Streamlit was chosen for the interface because it let the entire front end be written in plain Python alongside the rest of the system, and because Streamlit Community Cloud provided a genuinely free path to a real, public, deployed URL, which mattered a lot for making this feel like a finished product rather than something that only runs on one person's laptop.

Setting it up and running it locally

Clone the project's repository and move into its folder. Create a Python virtual environment and activate it, then install everything listed in the project's requirements file using pip.

Two credentials are required, both free to obtain. Create a file named exactly .env in the project's root folder. Inside it, add a Groq API key under the name GROQ_API_KEY, obtained from Groq's console, and a GitHub personal access token with repository write access under the name GITHUB_TOKEN, generated from your GitHub account's developer settings.

Once both are in place, start the app with the command streamlit run app.py. It will open automatically in your browser at a local address.

Using the app

In the sidebar, enter any public repository's owner and name, along with a real, currently existing issue number from that repository, then click Run Pipeline. The very first run against a brand new repository takes noticeably longer than later runs, because the system has to clone and embed that entire repository's code before anything else can happen. Every run after that against the same repository reuses the cached index and starts immediately.

Once a run completes, the interface shows a tab for each agent's output, the structured requirements, the investigation findings, the proposed fix and its code, the review verdict and feedback, the test results including the actual pytest output, and the generated

documentation. If the fix is flagged as needing review, an approval section appears at the bottom of the page. Clicking approve triggers the real GitHub actions described earlier, and the interface returns a genuine, clickable link to the pull request that was just opened.

Challenges faced and how they were solved

A fair amount of real debugging happened over the course of building this, and documenting it honestly feels more useful than pretending everything worked on the first try.

The most disruptive issue was a mid project model deprecation. The language model this whole system was originally built around, Llama 3.3 70B on Groq, stopped being available partway through development, breaking every single agent at once. The fix involved migrating every agent to Groq's officially recommended replacement, and then separately troubleshooting a new problem that migration introduced, occasional completely empty responses from the new model. That empty response issue turned out not to be random at all, it traced back to the new model being a reasoning model that silently spends part of its token budget on internal chain of thought before writing a visible answer, and without explicitly capping that reasoning effort, it could consume the entire response budget and leave nothing for the actual answer. Once that was identified and fixed by explicitly setting a low reasoning effort on every call, the empty response problem disappeared.

A separate structural bug showed up in the orchestration graph itself, where the Documentation agent had accidentally been wired to fire every single time the Code Review step ran, including on rejected attempts that were about to be retried, rather than only once review was genuinely settled. This meant a single pipeline run with two rejected revisions was silently generating documentation three times instead of once, burning through rate limits far faster than expected and making runs take much longer than they should have. Fixing this required restructuring that part of the graph so documentation and testing both only fire from a single point reached exactly once per run.

A more mundane but persistent issue was the interface allowing the same pipeline to be triggered twice in quick succession, which meant two full runs would execute concurrently, both competing for the same rate limited API key. This was fixed by explicitly disabling the run button while a pipeline is actively executing, backed by session state rather than relying on the button's default behavior.

Finally, the retrieval system originally only worked against one hard coded demo repository. Making it genuinely general purpose meant rebuilding it to clone and index whatever repository is entered at runtime, keyed into its own separate vector collection, so multiple repositories can be indexed and reused independently without conflicting with each other.

Testing and real world validation

Beyond normal development testing, this system was validated by actually running it against two different kinds of real repositories. First, against a well known, actively maintained open source project, where it produced a real, well formed pull request referencing the original issue. Second, against a personal project, where the resulting pull request was not just opened but genuinely reviewed and merged, which is the strongest confirmation available that the whole pipeline, investigation, fix, review, testing, documentation, and GitHub automation, actually works end to end on something real, not just in a controlled demo scenario.

Known limitations

The free tier of the Groq API imposes both daily and per minute token limits, so running the pipeline many times in quick succession can occasionally trigger a short, clearly surfaced rate limit delay rather than a silent failure. Free tier language models, even with the retry and repair logic built around them, occasionally still produce malformed or unexpected output, which is a real characteristic of working within a free tier rather than a flaw specific to this implementation. The tests the Testing Agent generates are deliberately self contained and illustrative, rather than wired into a target repository's own real test suite and dependencies, since fully installing and running a foreign project's actual environment was outside the scope of this version. Finally, an in progress pipeline run does not currently survive an application restart partway through, even though everything the system has already learned and stored up to that point does persist.

Possible future improvements

A natural next step would be full session persistence using something like a database backed checkpointer, allowing a pipeline run to be paused and resumed even across an application restart. Multi modal input, accepting something like a screenshot of a visual bug alongside its text description, would meaningfully extend the range of issues the system could investigate. Deeper integration with a target repository's actual existing test suite and dependencies, rather than generating illustrative standalone tests, would make the Testing Agent's output directly mergeable rather than purely demonstrative. And a persistent, per user history of past runs and approved fixes, beyond the current single session state, would make the tool more usable as an ongoing part of someone's actual workflow rather than a one run at a time demo.

Conclusion

This project set out to prove that a carefully built multi agent system, one that's genuinely grounded in real code, remembers its own past work, corrects itself when its own output isn't good enough, and never acts on a real repository without an explicit human decision, can meaningfully automate a real slice of the software engineering lifecycle. Not as a simulation or a proof of concept sitting untested on a laptop, but as something that produces actual, verifiable, mergeable results. A real pull request, opened entirely by this pipeline and approved by a human being, standing merged in a real repository, is the clearest possible evidence that it works.